OPTIONAL TASK

Name of Student : Anuja Vilas Wankhade

Domain: Artificial intelligence and machine Learning (AIML)

College Name: Manav School of Polytechnic AKOLA

Branch : Computer Engineering

GitHub Repository Link: https://github.com/anujaw193-star/python-project-collection

1. Import Libraries

```
[6]
✓ 0s
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

import joblib
```

2. Load Dataset

```
[7]
✓ 0s
df = pd.read_csv("cybersecurity_network_logs.csv")
```

3. Explore Dataset

```
[8]
print(df.head())
print(df.shape)
print(df.info())
print(df.describe())
print(df.isnull().sum())
```

```
   Protocol  Packet_Size_Bytes  ...  Geo_Distance_km  Is_Malicious
0       TCP                805  ...             1169             0
1       TCP                926  ...              379             0
2       TCP                974  ...               79             0
3      ICMP               6121  ...             1044             1
4       TCP                119  ...              788             0

[5 rows x 6 columns]
(25000, 6)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25000 entries, 0 to 24999
Data columns (total 6 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Protocol                25000 non-null  object
 1   Packet_Size_Bytes       25000 non-null  int64
 2   Connection_Duration_ms  25000 non-null  int64
 3   Failed_Logins           25000 non-null  int64
 4   Geo_Distance_km         25000 non-null  int64
 5   Is_Malicious            25000 non-null  int64
dtypes: int64(5), object(1)
memory usage: 1.1+ MB
None
```

```
None
       Packet_Size_Bytes  Connection_Duration_ms  ...  Geo_Distance_km  Is_Malicious
count        25000.00000            25000.000000  ...     25000.000000  25000.000000
mean           647.35788               99.912200  ...       641.442120      0.049880
std           1012.79147               99.374995  ...       981.653985      0.217701
min              0.00000                0.000000  ...         0.000000      0.000000
25%            148.00000               29.000000  ...       149.000000      0.000000
50%            357.00000               69.000000  ...       359.000000      0.000000
75%            736.00000              138.000000  ...       741.000000      0.000000
max          10032.00000             1213.000000  ...      8661.000000      1.000000

[8 rows x 5 columns]
Protocol                  0
Packet_Size_Bytes         0
Connection_Duration_ms    0
Failed_Logins             0
Geo_Distance_km           0
Is_Malicious              0
dtype: int64
```

4. Handle Missing Values

```
[9]
df = df.dropna()
```

5. Encode Categorical Columns

```
[10]
le = LabelEncoder()
for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])
```

6. Select Features and Target

```
[15]
X = df.drop('Is_Malicious', axis=1)
y = df['Is_Malicious']
```

7. Train-Test Split

```
[16]
✓ 0s
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

8. Feature Scaling

```
[17]
✓ 0s
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

9. Train Model

```
[20]
✓ 4s
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

RandomForestClassifier
RandomForestClassifier(random_state=42)
```

10. Prediction

```
[21]
✓ 0s
y_pred = model.predict(X_test)
```

11. Evaluation

```
[22]
✓ 0s
print("Accuracy :", accuracy_score(y_test, y_pred))

print(confusion_matrix(y_test, y_pred))

print(classification_report(y_test, y_pred))

Accuracy : 0.9964
[[4769    1]
 [  17  213]]
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      4770
           1       1.00      0.93      0.96       230

    accuracy                           1.00      5000
   macro avg       1.00      0.96      0.98      5000
weighted avg       1.00      1.00      1.00      5000
```

12. Save Model

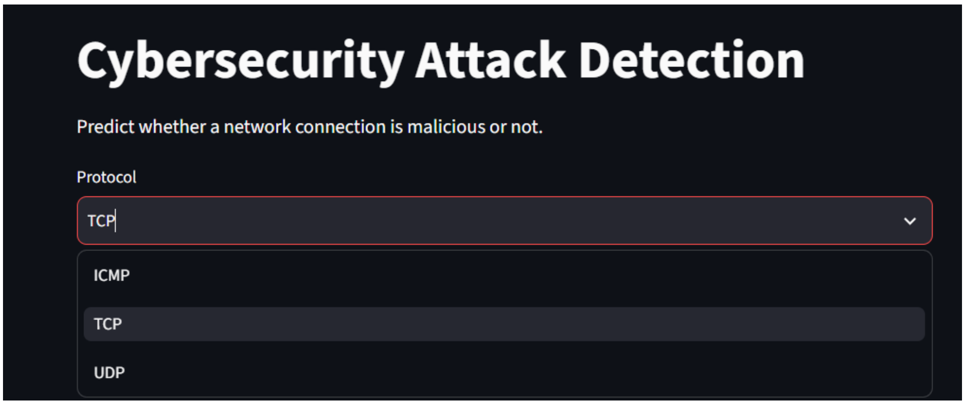
```
[23]
joblib.dump(model, "cyber_model.pkl")
joblib.dump(scaler, "scaler.pkl")
joblib.dump(X.columns.tolist(), "columns.pkl")

['columns.pkl']
```

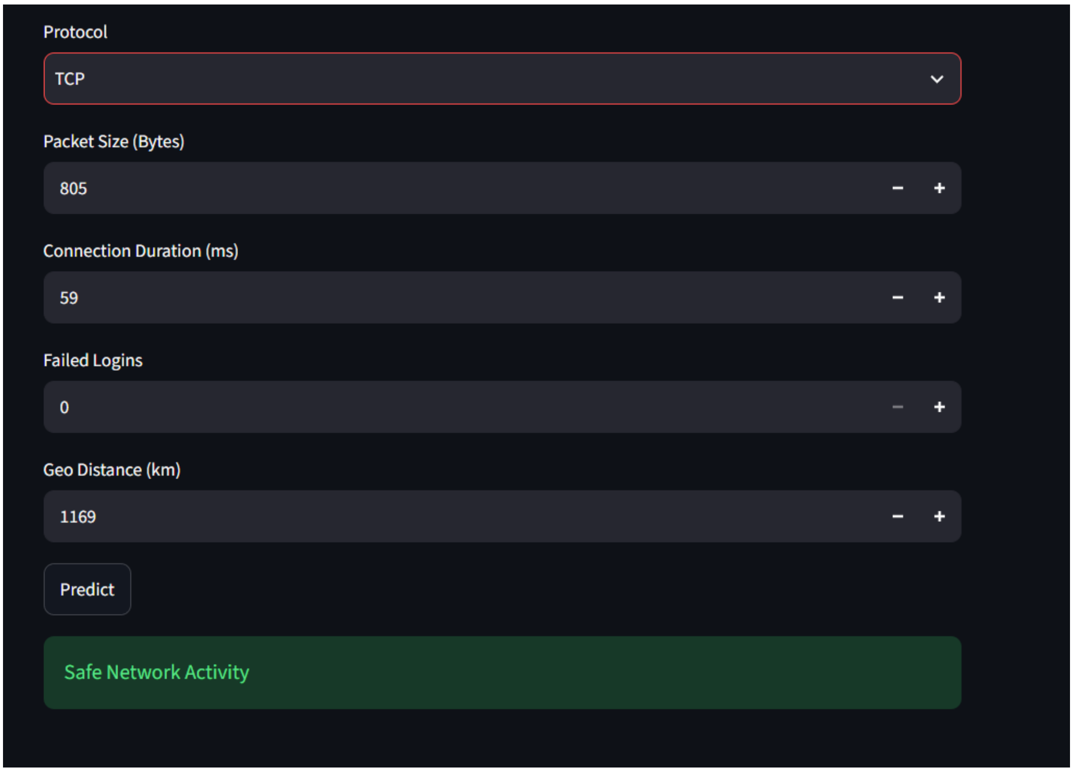
13. Frontend Code:

```
import streamlit as st
import pandas as pd
import joblib

# Load saved files
model = joblib.load("models/cyber_model.pkl")
scaler = joblib.load("models/scaler.pkl")
label_encoder = joblib.load("models/label_encoder.pkl")
columns = joblib.load("models/columns.pkl")

# Page Configuration
st.set_page_config(
    page_title="Cybersecurity Attack Detection",

    layout="centered"
)

st.title(" Cybersecurity Attack Detection")
st.write("Predict whether a network connection is malicious or not.")

# User Inputs
protocol = st.selectbox(
    "Protocol",
    label_encoder.classes_
)

packet_size = st.number_input(
    "Packet Size (Bytes)",
    min_value=0,
    value=500
)

connection_duration = st.number_input(
    "Connection Duration (ms)",
    min_value=0,
    value=100
)
```

```python
failed_logins = st.number_input(
    "Failed Logins",
    min_value=0,
    value=0
)

geo_distance = st.number_input(
    "Geo Distance (km)",
    min_value=0,
    value=300
)

# Prediction
if st.button("Predict"):

    protocol_encoded = label_encoder.transform([protocol])[0]

    input_data = pd.DataFrame([{
        "Protocol": protocol_encoded,
        "Packet_Size_Bytes": packet_size,
        "Connection_Duration_ms": connection_duration,
        "Failed_Logins": failed_logins,
        "Geo_Distance_km": geo_distance
    }])

    input_data = input_data[columns]

    input_scaled = scaler.transform(input_data)

    prediction = model.predict(input_scaled)[0]

    if prediction == 1:
        st.error(" Malicious Network Activity Detected")
    else:
        st.success(" Safe Network Activity")
```

Output:





Summary:

Dataset Title: Cybersecurity Network Attack Detection

- Selected a new dataset: cybersecurity_network_logs.csv
- Performed data preprocessing (checked missing values and encoded the Protocol column).
- Selected features (X) and target (y = Is_Malicious).
- Split the dataset into training and testing sets.
- Applied StandardScaler for feature scaling.
- Trained a Random Forest Classifier model.
- Evaluated the model using Accuracy, Confusion Matrix, and Classification Report.
- Saved the trained model and preprocessing files using Joblib.
- Created a Streamlit frontend to allow users to enter network details and predict whether the network activity is Safe or Malicious.